

Data Handling: Import, Cleaning and Visualisation

Exercise/Workshop 3: Programming with Data

Ulrich Matter

Exercises

Exercise A: Write a Sum Function

In the lecture on Programming with Data, we have discussed the most basic programming concepts in R. Among other things, you have learned how we can use for-loops to sum up numbers stored in a numeric vector. Use the code example in the respective lecture notes and the following empty function construct to implement a function that takes a numeric vector as input and returns the sum of the vector's elements.

```
my_sum <-  
  function(x){  
  
  }
```

Test your function by comparing its output with the output of `sum()` (the pre-implemented sum function in R) when calling it with the same input.

Exercise B: Robustness and Warnings

Once you have implemented and successfully tested `my_sum()`, see what happens if you call it with the following vector as input

```
numbers2 <- c("1", "2", "3")
```

Why do we get an error? Investigate by comparing the vector `numbers` from the code example above with the new vector `numbers2` (the functions `str()` and `class()` might help you).

Extend your `my_sum()` function with a control statement that checks whether the input is of the right class for the function to work (Hint: `class(x)!="numeric"`). The goal is to make the function more robust to wrong input. That is, if the input is wrong, issue a warning but don't break down with an error. Use the following code for the warning message:

```
warning("Wrong input! This function only accepts numeric or integer values!")
```

Test your enhanced sum function with the two input vectors `numbers` and `numbers2`.

Exercise C: Standard Deviation Function

During the lecture on Programming with Data we have step-by-step implemented our own R function to compute the mean. Now implement an R function called `my_sd` that computes the standard deviation, defined as: $SD = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$. (Hint: revisit the math operators section in the notes on the first exercise session: https://umatter.github.io/courses/datahandling/notes/html/E01_tools_textfiles.html).

Test your implementation by comparing its output with the already implemented `sd()` function.

Mock Exam Questions

Part 1: TRUE/FALSE Questions

For each of the following statements, decide whether the statement is TRUE (T) or FALSE (F).

1. $1\text{GB} = 1000 \times 1000 \times 1000 \times 16 \text{ bits}$
-

Part 2: Multiple Choice Type A

For each of the following questions/statements, decide which of the possible responses are correct. NOTE: Either ONE OR SEVERAL of the response statements can be correct. In order to get a point, all correct response statements need to be marked!

1. In csv files, rows of data are indicated by new lines. Therefore...

- A) CSV files can only be opened in spreadsheet software like Excel.
 - B) The underlying binary code contains white spaces whenever an end of line occurs
 - C) Computers recognize the end of the line because each line contains the same amount of characters
 - D) A special character marks the end of a line. This is why a CSV-parser recognizes when a line ends.
-

Part 3: Multiple Choice Type B (only one correct!)

For each of the following questions/statements, decide which of the possible responses is correct. NOTE: For each question only ONE of the possible responses is correct. Select ONLY ONE of the response statements per question!

1. Look at the following line of code. Which of the four statements about the code is correct?

```
<html><head><title>alpha</title></head><head><body>beta</body></head></html>
```

- A) 'alpha' and 'beta' are HTML-tag attributes
- B) The <title> tag should be placed in between the <body> tags in order to make this code a valid HTML document
- C) The code represents a basically correctly specified HTML-document
- D) All tags occurring in the code are valid HTML tags